МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №1
по дисциплине «Объектно-ориентированное программирование»
Тема: «Создания игрового приложения. Часть первая»

Студент гр. 4388 _____ Кузьмин Г.И.

Преподаватель _____ Жангиров Т.Р.

Санкт-Петербург
2025

Цель работы.

Разработка игрового приложения на языке C++ с использованием объектно-ориентированного подхода. Реализация перемещения игрока по карте и сражения с врагами.

Задание.

1. Создать класс игрока, который должен хранить информацию об игроке (его жизни, урон, очки, и т.д. - студент сам определяет необходимые для работы характеристики). Объект класса игрока должен перемещаться по карте. Если у игрока кончаются жизни, то происходит конец игры.
2. Создать класс врага, который хранит параметры жизней и урона. Объектами класса врага управляет компьютер. При перемещении, если враг пытается перейти на клетку с игроком, то перемещение не происходит, и игроку наносится урон.
3. Создать класс квадратного/прямоугольного игрового поля, по которому перемещаются игрок и враги. Игровое поле не должно быть меньше 10 на 10 клеток, и не больше 25 на 25 клеток. Размеры поля задаются через конструктор. Рекомендуется для хранения информации об отдельных клетках поля создать отдельный класс.
4. Реализовать конструкторы перемещения и копирования для поля, а также соответствующие операторы присваивания с копированием и перемещением (должна происходить глубокая копия).
5. Реализовать непроходимые клетки на поле. При попытке врагов или игрока перейти на такую клетку, перемещение не происходит. Заполнение поля непроходимыми клетками происходит в момент создания поля.
6. Добавить возможность для игрока переключаться на ближний или дальний бой с изменением значения наносимого урона. Такое переключение требует один ход.

Примечания:

- Не забывайте для полей и методов определять модификаторы доступа
- Для обозначения переменной, которая принимает небольшое ограниченное количество значений, используйте enum
- Не используйте глобальные переменные
- При реализации перемещения не должно быть лишнего копирования
- У поля не должно быть методов, возвращающих указатель на поле в явном виде, так как это небезопасно
- Информация о координатах игрока не должна дублироваться у игрока и у поля

Выполнение работы.

- Класс Player – класс, представляющий игрока, которым управляет пользователь. Инкапсулирует состояние персонажа, его характеристики и логику смены боевых режимов.

Поля класса:

1. int hp – текущие очки здоровья игрока. Значение не может быть меньше нуля.
2. int damage – урон, наносимый игроком. Зависит от текущего режима боя.
3. int score – количество набранных очков за игру.
4. std::pair<int, int> position – текущие координаты игрока на поле (x, y).
5. CombatMode mode – текущий режим боя (ближний MELEE или дальний RANGED).

Методы класса:

1. Player(int startHp) – конструктор, устанавливает начальное здоровье, позицию (1,1) и режим MELEE.
2. int getHp() const – возвращает текущее количество здоровья.
3. int getDamage() const – возвращает урон игрока.
4. int getScore() const – возвращает количество очков.

5. std::pair<int, int> getPosition() const – возвращает пару координат (x,y).
6. CombatMode getMode() const – возвращает текущий режим боя (MELEE/RANGED).
7. void takeDamage(int amount) – уменьшает hp на amount (с проверкой на 0).
8. void addScore(int points) – увеличивает счет игрока.
9. void setPosition(int x, int y) – обновляет координаты игрока.
10. void switchMode() – переключает режим боя и меняет значение damage.

- Класс Enemy – класс, описывающий противника. Реализует простейшую логику для поиска игрока.

Поля класса:

1. Int hp – очки здоровья врага.
2. Int damage – урон, наносимый врагом.
3. Int x, y – текущие координаты на поле.

Методы класса:

1. Enemy(int hp, int damage, int startX, int startY) – конструктор для инициализации всех параметров.
2. int getX() const – возвращает координату по горизонтали.
3. int getY() const – возвращает координату по вертикали.
4. void moveTowardsPlayer(Player& player, const GameField& field) – метод поиска игрока: рассчитывает шаг в сторону игрока, проверяет препятствия и наносит урон, если игрок вплотную.

- Класс GameCell – класс, представляющий возможные типы клеток для игры.

Методы класса:

1. GameCell() – конструктор по умолчанию (пустая клетка).
2. void setType(CellType newType) – установка типа клетки.
3. CellType getType() const – получение типа клетки.

- Класс GameField – класс, управляющий сеткой клеток и отвечает за корректное копирование данных (динамической памяти).

Поля класса:

1. int width, height – размеры поля.
2. std::vector<std::vector<GameCell>> cells – сетка объектов ячеек.

Методы класса:

1. GameField(int w, int h) – конструктор: задает размер и случайно генерирует стены #.
2. GameField(const GameField& other) – конструктор копирования: создает глубокую копию данных другого поля.
3. GameField(GameField&& other) noexcept – конструктор перемещения: эффективно забирает данные у временного объекта.
4. GameField& operator=(const GameField& other) – оператор копирующего присваивания: глубокое копирование при присваивании.
5. GameField& operator=(GameField&& other) noexcept – оператор перемещающего присваивания.
6. int getWidth() const – возвращает ширину поля.
7. int getHeight() const – возвращает высоту поля.
8. const GameCell& getCell(int x, int y) const – возвращает ссылку на клетку для чтения.
9. void setCellType(int x, int y, CellType type) – меняет тип конкретной клетки.
10. bool isObstacle(int x, int y) const – проверяет, является ли клетка непроходимой (с учетом границ поля).

- Класс Game – главный класс, объединяющий все части проекта в единый игровой цикл.

Поля класса:

1. GameField field – объект поля.
2. Player player – объект игрока.
3. std::vector<Enemy> enemies – список врагов.

4. bool isRunning – статус цикла игры.

Методы класса:

1. Game() – конструктор: создает поле, игрока и расставляет врагов.
2. void render() – выводит игровое состояние в консоль.
3. void handleInput() – считывает команды игрока и запускает ход ИИ.
4. void update() – проверяет, не закончились ли у игрока жизни.
5. void run() – запускает бесконечный цикл while(isRunning).

Далее приведена UML – диаграмма классов для данной лабораторной работы, отражающая связи между классами.

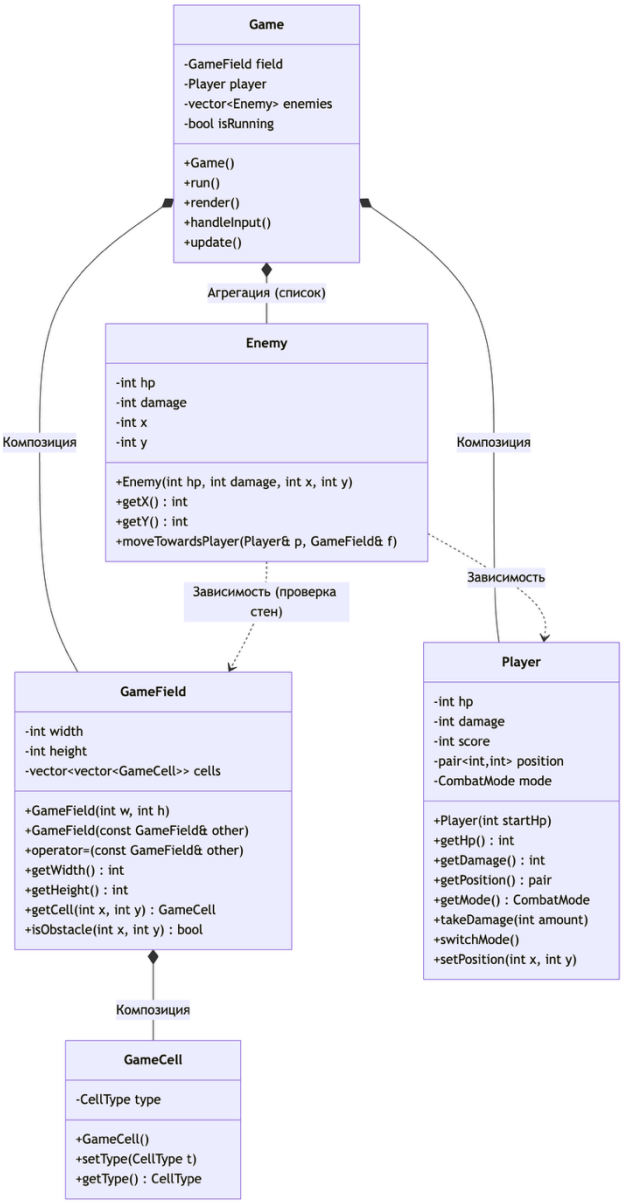

Рис 1. – UML-диаграмма классов

Тестирование.

1. Состояние игры сразу после запуска приложения.

```
=== BATTLE FIELD ===
HP: 100 | Mode: MELEE
Controls: WASD - move, M - switch mode, Q - quit
--------------------
. . . . . . # . . . . . . # .
. P # . . . # . . . . # . . .
. . . . . . . # . . # . . . .
. . . # # . . . . . . # . . .
. . . . # . . . # # . . . . .
. . . # . E . . . . . . . . #
. . . . . . . . . . . . # . .
. . . . . . . . # . # . . . .
. . . . . . . . . . . . . . .
. . . . . . . . . . . . . . #
. . E . . . . . . . . . . . #
. . . . . . . . . . . . # . .
. . # . . # # . . . . . E . .
. # . . . . . # . . . . . . .
. . . . . . . . . . . . . . .
Enter command:
```

Рис 2. – Запуск приложения

2. Проверка прохождения через непроходимую клетку

```
=== BATTLE FIELD ===
HP: 100 | Mode: MELEE
Controls: WASD - move, M - switch mode, Q - quit
--------------------
. . . . . . # . . . . . . # .
. P # . . . # . . . . # . . .
. . . . . . . # . . # . . . .
. . . # # . . . . . . # . . .
. . . . # . . . # # . . . . .
. . . # . E . . . . . . . . #
. . . . . . . . . . . . # . .
. . . . . . . . # . # . . . .
. . . . . . . . . . . . . . .
. . . . . . . . . . . . . . #
. . E . . . . . . . . . . . #
. . . . . . . . . . . . # . .
. . # . . # # . . . . . E . .
. # . . . . . # . . . . . . .
. . . . . . . . . . . . . . .
Enter command: d
You hit a wall!
```

Рис 3. – Непроходимая клетка

3. Получение урона от врагов.

```
HP: 100 | Mode: MELEE
Controls: WASD - move, M - switch mode, Q - quit
--------------------
. . . . . . . . . . . . . . .
. . . # . . . . . . # . . . . .
. # . . . . . # . . # # . . .
. . P . . . . . . . # # . . .
. . . . # . # . # . . # . . #
. . E . . . . . . # . . . . #
. # . . . . . . . # # # . . #
. . E . . . # . . . . . . . .
. . . # . . . # . . . . # . #
. . . . . . . . # . . . . . .
. . . . . . . . . . . . . . .
. . . . # . . . . . . # . # .
. . . . . . # . . E . . . . .
. . . . . . . . . . # . . . .
. . . # . . . # . . . . . . .
Enter command: s
=== BATTLE FIELD ===
HP: 90 | Mode: MELEE
Controls: WASD - move, M - switch mode, Q - quit
--------------------
```

Рис. 4 – Получение урона от врага

4. Переключение режима боя.

```
=== BATTLE FIELD ===
HP: 100 | Mode: MELEE
Controls: WASD - move, M - switch mode, Q - quit
--------------------
. . # . . . . . . . . . . . .
. P . . . . . . . . # . . . .
. # . . . . . . . . . . . # .
. . . . . . . . . . . . . . .
. . . # . # . . . . . . . . .
. # . . . E . . . # . . . . .
. . . . . . # . . . . . . . #
. . . . . . . . . . . . . . .
. . . . . . . . . . # . . . .
# . . . . . . # . . . # . . .
. # E . . . . . . # . . . . .
. . . . . . . . . # # . . . .
. . # . . . . . . . # # E # .
. . . . . . . # . # # . . . .
# . # . . . . . . . . . . . .
Enter command: m
Switched combat mode!
=== BATTLE FIELD ===
HP: 100 | Mode: RANGED
Controls: WASD - move, M - switch mode, Q - quit
```

Рис. 5 – Переключение режима боя

5. Окончание игры. Здоровье достигло нуля.

```
=== BATTLE FIELD ===
HP: 0 | Mode: RANGED
Controls: WASD - move, M - switch mode, Q - quit
--------------------
. . # . . . . . . . . . . . .
. . . . . . . . . . # . . . .
. # . . . . . . . . . . . # .
. . . . . . . . . . . . . . .
. . E # . # . . . . . . . . .
. # P . . . . . . # . . . . .
. . . . . . # . . . . . . . #
. . . . . . . . . . . . . . .
. . . . . . . . . . # . . . .
# . . . . . . # . . . # . . .
. # . . . . . . . # . . . . .
. . . . . . . . . # # . . . .
. . # . . . . . . . # # E # .
. . . . . . . # . # # . . . .
# . # . . . . . . . . . . . .

!!! GAME OVER !!!
The enemies have defeated you.
```

Рис 6. – Окончание игры

Вывод

В ходе выполнения работы было разработано игровое приложения на языке C++ с использованием объектно-ориентированного подхода. Реализация перемещения игрока по карте и сражения с врагами. Были реализованы непроходимые клетки и логика поведения врагов. Тестирование показало, что программа работает корректно.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: main.cpp

```
#include "Game.h"

int main() {
    Game game;

    game.run();

    return 0;
}
```

Название файла: Player.h

```
#ifndef PLAYER_H
#define PLAYER_H

#include "Enums.h"
#include <utility>

class Player {
private:
    int hp;
    int damage;
    int score;
    std::pair<int, int> position;
    CombatMode mode;

public:
    Player(int startHp);

    int getHp() const;
    int getDamage() const;
    int getScore() const;
    std::pair<int, int> getPosition() const;
    CombatMode getMode() const;

    void takeDamage(int amount);
    void addScore(int points);
    void setPosition(int x, int y);

    void switchMode();
};

#endif
```

Название файла: Player.cpp

```
#include "Player.h"

Player::Player(int startHp)
    : hp(startHp), damage(15), score(0), mode(CombatMode::MELEE) {
    position = {1, 1};
}
```

```
int Player::getHp() const {
    return hp;
}

int Player::getDamage() const {
    return damage;
}

int Player::getScore() const {
    return score;
}

std::pair<int, int> Player::getPosition() const {
    return position;
}

CombatMode Player::getMode() const {
    return mode;
}

void Player::takeDamage(int amount) {
    hp -= amount;
    if (hp < 0) hp = 0;
}

void Player::addScore(int points) {
    if (points > 0) score += points;
}

void Player::setPosition(int x, int y) {
    position = {x, y};
}

void Player::switchMode() {
    if (mode == CombatMode::MELEE) {
        mode = CombatMode::RANGED;
        damage = 5;
    } else {
        mode = CombatMode::MELEE;
        damage = 15;
    }
}
```

Название файла: Enemy.h

```
#ifndef ENEMY_H
#define ENEMY_H

#include <utility>

class GameField;
class Player;

class Enemy {
private:
    int hp;
    int damage;
```

```
    int x, y;

public:
    Enemy(int hp, int damage, int startX, int startY);
    int getX() const { return x; }
    int getY() const { return y; }

    void moveTowardsPlayer(Player& player, const GameField& field);
};

#endif
```

Название файла: Enemy.cpp

```
#include "Enemy.h"
#include "Player.h"
#include "GameField.h"

Enemy::Enemy(int hp, int damage, int startX, int startY)
    : hp(hp), damage(damage), x(startX), y(startY) {}

void Enemy::moveTowardsPlayer(Player& player, const GameField& field)
{
    int pX = player.getPosition().first;
    int pY = player.getPosition().second;

    int nextX = x;
    int nextY = y;

    if (x < pX) nextX++;
    else if (x > pX) nextX--;
    else if (y < pY) nextY++;
    else if (y > pY) nextY--;

    if (nextX == pX && nextY == pY) {
        player.takeDamage(damage);
        return;
    }

    if (field.isObstacle(nextX, nextY)) {
        return;
    }

    x = nextX;
    y = nextY;
}
```

Название файла: Enums.h

```
#ifndef ENUMS_H
#define ENUMS_H

enum class Direction { UP, DOWN, LEFT, RIGHT };
enum class CellType { EMPTY, OBSTACLE, SLOW };
enum class CombatMode { MELEE, RANGED };

#endif
```

Название файла: Game.h

```
#ifndef GAME_H
#define GAME_H

#include <vector>
#include "GameField.h"
#include "Player.h"
#include "Enemy.h"
#include <iostream>

class Game {
private:
    GameField field;
    Player player;
    std::vector<Enemy> enemies;
    bool isRunning;

    void render();
    void handleInput();
    void update();

public:
    Game();
    void run();
};

#endif
```

Название файла: Game.cpp

```
#include "Game.h"
#include "Player.h"
#include "Enemy.h"
#include "GameField.h"
#include "GameCell.h"
#include "Enums.h"
#include <iostream>
#include <cstdlib>

Game::Game() : field(15, 15), player(100), isRunning(true) {
    player.setPosition(1, 1);


    enemies.push_back(Enemy(50, 10, 5, 5));
    enemies.push_back(Enemy(50, 10, 12, 12));
    enemies.push_back(Enemy(50, 10, 2, 10));
}

void Game::render() {
    system("clear");

    std::cout << "=== BATTLE FIELD ===" << std::endl;
    std::cout << "HP: " << player.getHp() << " | Mode: "
              << (player.getMode() == CombatMode::MELEE ? "MELEE" :
"RANGED") << std::endl;
    std::cout << "Controls: WASD - move, M - switch mode, Q - quit"
<< std::endl;
    std::cout << "--------------------" << std::endl;
```

```
        for (int y = 0; y < field.getHeight(); ++y) {
            for (int x = 0; x < field.getWidth(); ++x) {


                if   (x   ==   player.getPosition().first   &&   y   ==
player.getPosition().second) {
                    std::cout << "P ";
                    continue;
                }

                bool enemyDrawn = false;
                for (const auto& e : enemies) {
                    if (e.getX() == x && e.getY() == y) {
                        std::cout << "E ";
                        enemyDrawn = true;
                        break;
                    }
                }
                if (enemyDrawn) continue;


                if (field.getCell(x, y).getType() == CellType::OBSTACLE)
{
                    std::cout << "# ";
                } else {
                    std::cout << ". ";
                }
            }
            std::cout << std::endl;
        }
    }

    void Game::handleInput() {
        char input;
        std::cout << "Enter command: ";
        std::cin >> input;

        bool turnTaken = false;

        if (input == 'q') {
            isRunning = false;
            return;
        }

        if (input == 'm') {
            player.switchMode();
            std::cout << "Switched combat mode!" << std::endl;
            turnTaken = true;
        } else if (input == 'w' || input == 's' || input == 'a' || input
== 'd') {
            int nextX = player.getPosition().first;
            int nextY = player.getPosition().second;

            if (input == 'w') nextY--;
            if (input == 's') nextY++;
            if (input == 'a') nextX--;
            if (input == 'd') nextX++;
```

```
            if (nextX >= 0 && nextX < field.getWidth() && nextY >= 0 &&
nextY < field.getHeight()) {
                if (!field.isObstacle(nextX, nextY)) {
                    player.setPosition(nextX, nextY);
                    turnTaken = true;
                } else {
                    std::cout << "You hit a wall!" << std::endl;
                }
            }
        }


        if (turnTaken) {
            for (auto& e : enemies) {
                e.moveTowardsPlayer(player, field);
            }
        }
    }


    void Game::update() {
        if (player.getHp() <= 0) {
            isRunning = false;
            render();
            std::cout << "\n!!! GAME OVER !!!" << std::endl;
            std::cout << "The enemies have defeated you." << std::endl;
        }
    }

    void Game::run() {
        while (isRunning) {
            render();
            handleInput();
            update();
        }
    }
```

Название файла: GameCell.h

```
#ifndef GAMECELL_H
#define GAMECELL_H

#include "Enums.h"

class GameCell {
private:
    CellType type;
public:
    GameCell();
    void setType(CellType newType);
    CellType getType() const;
};

#endif
```

Название файла: GameCell.cpp

```
#include "GameCell.h"
```

```
GameCell::GameCell() : type(CellType::EMPTY) {}

void GameCell::setType(CellType newType) {
    type = newType;
}

CellType GameCell::getType() const {
    return type;
}
```

Название файла: GameField.h

```
#ifndef GAMEFIELD_H
#define GAMEFIELD_H

#include <vector>
#include "GameCell.h"

class GameField {
private:
    int width, height;
    std::vector<std::vector<GameCell>> cells;

public:
    GameField(int w, int h);

    GameField(const GameField& other);
    GameField(GameField&& other) noexcept;
    GameField& operator=(const GameField& other);
    GameField& operator=(GameField&& other) noexcept;
    ~GameField() = default;

    int getWidth() const { return width; }
    int getHeight() const { return height; }

    const GameCell& getCell(int x, int y) const;
    void setCellType(int x, int y, CellType type);
    bool isObstacle(int x, int y) const;
};

#endif
```

Название файла: GameField.cpp

```
#include "GameField.h"
#include <ctime>
#include <cstdlib>

GameField::GameField(int w, int h) {
    width = (w < 10) ? 10 : (w > 25 ? 25 : w);
    height = (h < 10) ? 10 : (h > 25 ? 25 : h);
    cells.resize(height, std::vector<GameCell>(width));

    std::srand(std::time(0));
    for(int y = 0; y < height; ++y) {
        for(int x = 0; x < width; ++x) {
            if (std::rand() % 8 == 0)
                cells[y][x].setType(CellType::OBSTACLE);
        }
```

```
        }
        cells[1][1].setType(CellType::EMPTY);
    }

    bool GameField::isObstacle(int x, int y) const {
        if (x < 0 || x >= width || y < 0 || y >= height) return true;
        return cells[y][x].getType() == CellType::OBSTACLE;
    }

    const GameCell& GameField::getCell(int x, int y) const { return
cells[y][x]; }
    void GameField::setCellType(int x, int y, CellType type)
{ cells[y][x].setType(type); }

    GameField::GameField(const GameField& other) : width(other.width),
height(other.height), cells(other.cells) {}
    GameField::GameField(GameField&& other) noexcept :
width(other.width), height(other.height), cells(std::move(other.cells)) {}
    GameField& GameField::operator=(const GameField& other) {
        if (this != &other) { width = other.width; height = other.height;
cells = other.cells; }
        return *this;
    }
    GameField& GameField::operator=(GameField&& other) noexcept {
        if (this != &other) { width = other.width; height = other.height;
cells = std::move(other.cells); }
        return *this;
    }
```

Название файла: Makefile

```
CXX = g++
CXXFLAGS = -Wall -Wextra -std=c++17

TARGET = game

OBJS = main.o Game.o GameField.o Player.o Enemy.o GameCell.o

all: $(TARGET)

$(TARGET): $(OBJS)
     $(CXX) $(CXXFLAGS) -o $(TARGET) $(OBJS)

main.o: main.cpp Game.h
     $(CXX) $(CXXFLAGS) -c main.cpp

Game.o: Game.cpp Game.h GameField.h Player.h Enums.h
     $(CXX) $(CXXFLAGS) -c Game.cpp

GameField.o: GameField.cpp GameField.h GameCell.h
     $(CXX) $(CXXFLAGS) -c GameField.cpp

Player.o: Player.cpp Player.h Enums.h
     $(CXX) $(CXXFLAGS) -c Player.cpp

Enemy.o: Enemy.cpp Enemy.h
     $(CXX) $(CXXFLAGS) -c Enemy.cpp
```

```
GameCell.o: GameCell.cpp GameCell.h Enums.h
	$(CXX) $(CXXFLAGS) -c GameCell.cpp

clean:
	rm -f $(OBJS) $(TARGET)
```